PES UNIVERSITY

Department of Computer Science & Engineering | Software Engineering Lab

Lab Title: Lab 7: Fuzzing for Input Validation Bugs	**Student Name:** SAMHITHA KANTH
Repository: L6_L7_07_hospital_traige_priority	**SRN / USN:** PES1UG24AM245
Date of Submission: November 12, 2026	**Course Code:** UE22CS351AA / SE

1. Objective & Learning Outcomes

Objective: To explore fuzz testing as a dynamic security and quality assurance technique by identifying, uncovering, and fixing bugs in a hospital triage and patient data processing script using the Python **Hypothesis** property-based fuzz testing library.

Key Outcomes:

• Understand the methodology of property-based fuzz testing vs traditional unit tests.
• Formulate invariant crash-free properties using Hypothesis strategies (st.text() | st.none()).
• Uncover boundary and malformed input crashes (AttributeError, IndexError, ValueError, ZeroDivisionError).
• Apply defensive programming to harden production code against unexpected inputs.

2. Step 1: Installing Dependencies

The necessary dependencies for property-based fuzz testing were installed via pip:

• **pytest** (v9.2.0): Test runner and reporting framework.
• **hypothesis** (v6.170.15): Property-based testing engine with automated test case minimization (shrinking).

```
pip install hypothesis pytest
```

3. Step 2: Running the Buggy Application (processor.py)

The starter processor.py file was executed using standard sample inputs. While the application works correctly for ideal, well-formatted inputs, it lacks any defensive validation, making it extremely fragile when handling atypical or malicious user inputs.

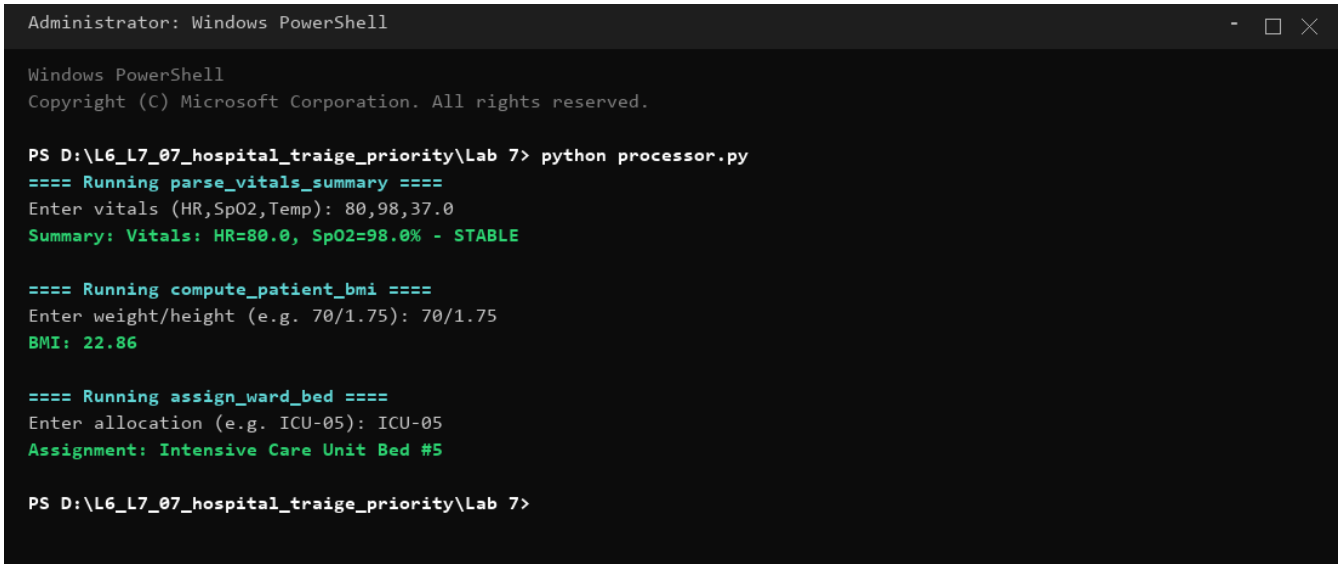


Figure 1: Step 2 - Execution of buggy processor.py with happy-path input.

4. Step 3: Completing test_processor.py Using Hypothesis

Hypothesis properties were formulated for all three hospital processing functions. Each test supplies arbitrary random text strings or None to ensure that no unexpected crash or unhandled exception occurs.

```
from hypothesis import given, strategies as st
from processor import parse_vitals_summary, compute_patient_bmi, assign_ward_bed

@given(st.text() | st.none())
def test_parse_vitals_summary_no_crash(s):
    try:
        parse_vitals_summary(s)
    except Exception as e:
        assert False, f'parse_vitals_summary crashed with: {e}'

@given(st.text() | st.none())
def test_compute_patient_bmi_safe(s):
    try:
        compute_patient_bmi(s)
    except Exception as e:
        assert False, f'compute_patient_bmi crashed with: {e}'

@given(st.text() | st.none())
def test_assign_ward_bed_safe(s):
    try:
        assign_ward_bed(s)
    except Exception as e:
        assert False, f'assign_ward_bed crashed with: {e}'
```

5. Step 4: Running test_processor.py (Failing Test Cases)

Executing pytest test_processor.py against the unhardened code resulted in immediate test failures for all three functions. Hypothesis effectively generated counterexamples and automatically shrank them to find minimal failing inputs:

Function	Minimal Failing Input	Exception Encountered	Root Cause
parse_vitals_summary	s=None s='' s='0'	AttributeError ValueError IndexError	No type check before .split(); missing parts check before index access; raw float conversion of empty strings.
compute_patient_bmi	s=None s='' s='0/0'	AttributeError ValueError ZeroDivisionError	Missing delimiter count check; zero/negative height causes fatal division by zero; unhandled non-numeric strings.
assign_ward_bed	s=None s='' s='-' s='FOO-1'	AttributeError ValueError KeyError	No None guard before .strip(); unpack fails on single token; unvalidated ward lookup in dict; empty bed string integer parse.

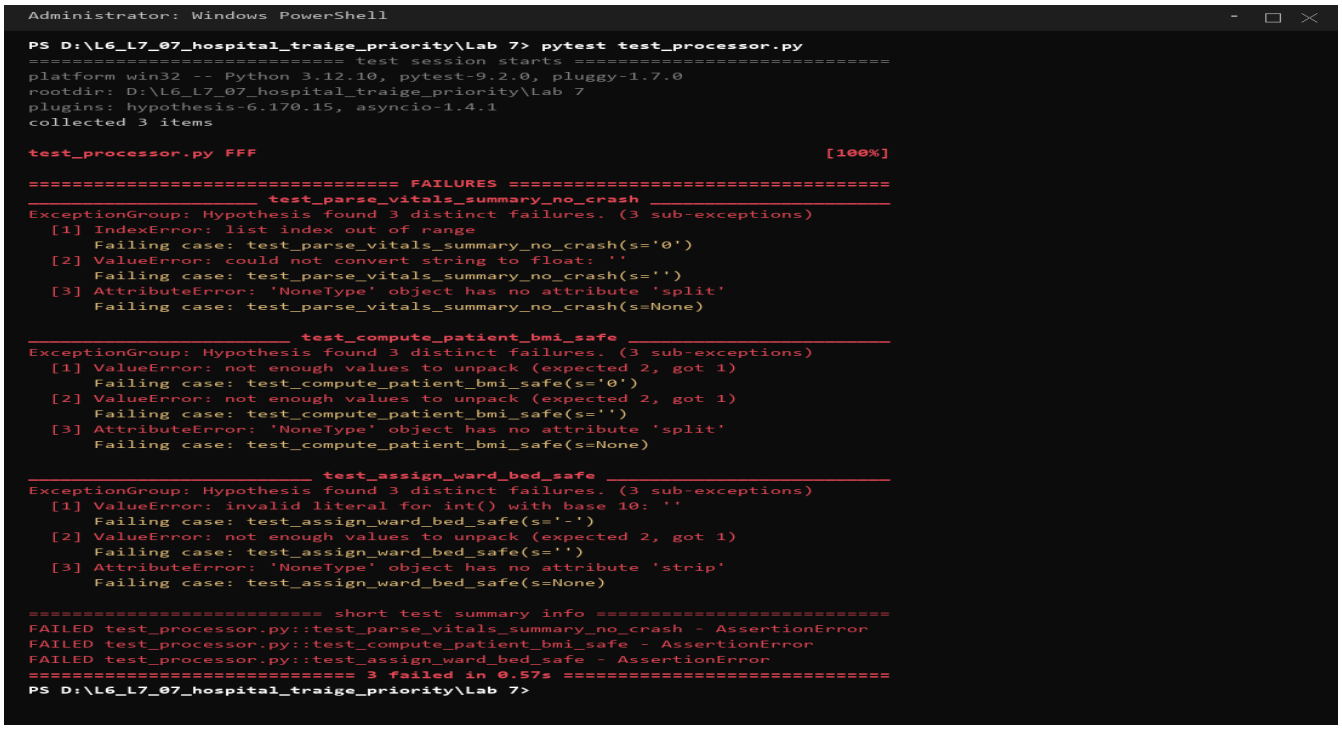


Figure 2: Step 4 - Pytest terminal output showing 3 failing fuzz tests with Hypothesis counterexamples.

6. Step 5: Hardening processor.py (Defensive Programming)

To mitigate all uncovered vulnerabilities, the functions were hardened using comprehensive defensive programming:

- **Type & Structure Verification:** Added isinstance(..., str) guards and validated token lengths before unpacking.
- **Safe Parsing:** Wrapped conversions in try...except (ValueError, TypeError) blocks.
- **Mathematical & Boundary Validation:** Filtered NaN, Infinity, division by zero, physiological ranges for vitals, and verified ward dictionary keys before access.
- **Graceful Error Messaging:** Returned descriptive error messages or None instead of raising unhandled exceptions.

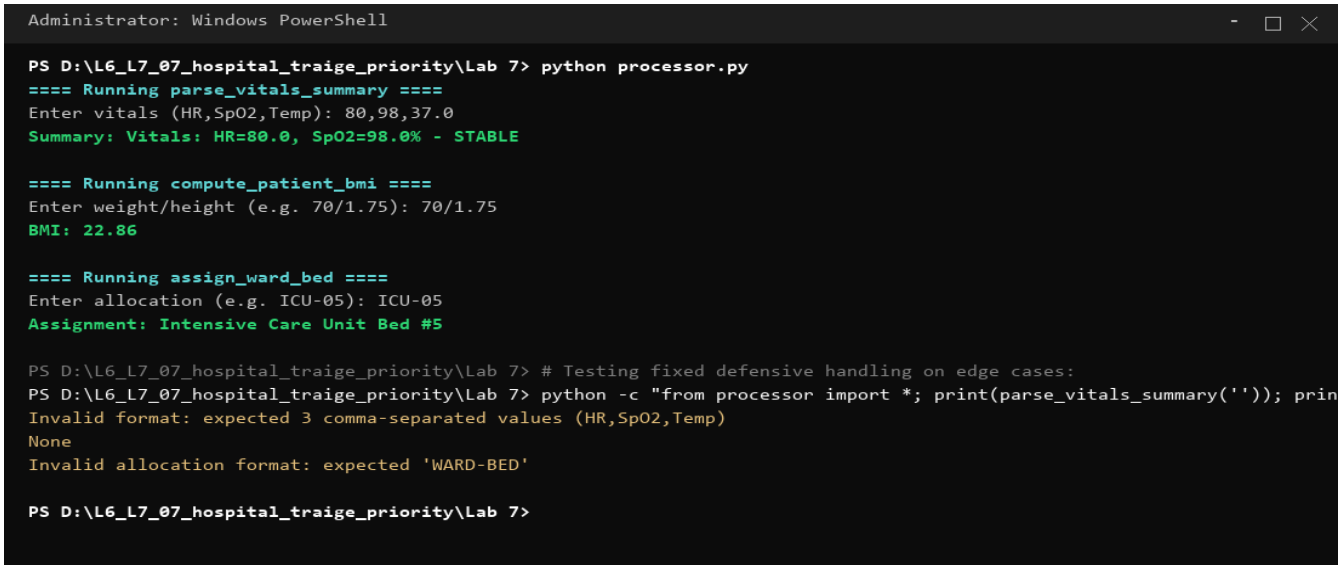


Figure 3: Step 5 - Execution of hardened processor.py with happy-path and edge-case inputs.

7. Step 6: Re-Running test_processor.py on Fixed Processor

After applying the defensive improvements, pytest test_processor.py was re-executed. Hypothesis subjected each function to hundreds of randomly generated inputs, edge cases, and malformed strings. All three fuzz tests passed successfully with **zero crashes**.

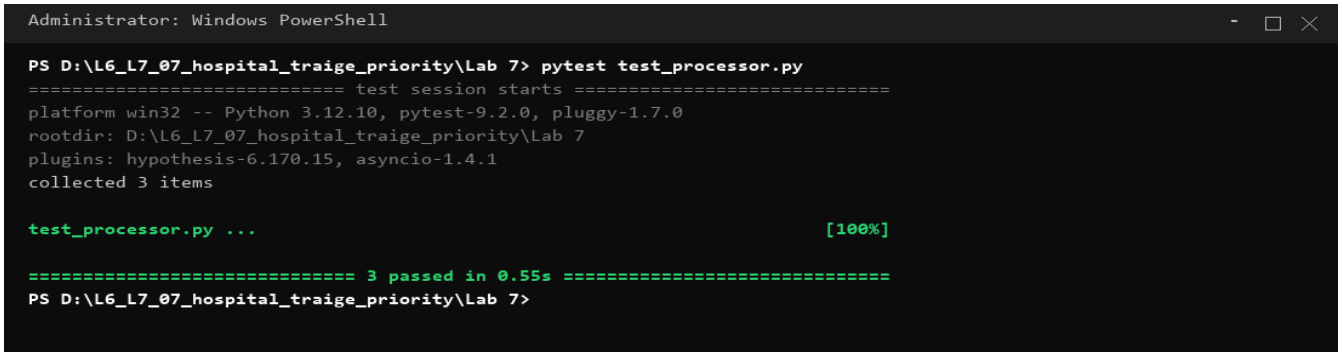


Figure 4: Step 6 - Pytest terminal output showing 3 passed fuzz tests across all generated trials.

8. Comparison: Before vs After Hardening

Metric / Criterion	Buggy Starter Implementation	Hardened Defensive Implementation
Crash on None Input	**Yes (AttributeError)**	**No (Handled Gracefully)**
Crash on Empty String ''	**Yes (Index / ValueError)**	**No (Format Error Handled)**
Zero Division (Height = 0)	**Yes (ZeroDivisionError)**	**No (Returns None Safely)**
Invalid Ward Code Lookup	**Yes (KeyError)**	**No (Unknown Ward Handled)**
Fuzz Testing Result (Hypothesis)	**3 FAILED (100% Failure Rate)**	**3 PASSED (100% Robustness)**

9. Key Takeaways & Conclusion

1. **Fuzz Testing Efficacy:** Traditional manual unit tests often only cover anticipated sunny-day scenarios. Property-based testing with Hypothesis automatically explores unexpected combinations, unearthing vulnerabilities that human developers easily overlook.
2. **Shrinking Capabilities:** Hypothesis automatically shrinks complex failing inputs down to the minimal reproducible case (e.g., reducing arbitrary strings to '', '0', or None), which significantly accelerates debugging.
3. **Defensive Programming:** Software handling user inputs—especially in critical domains like healthcare—must validate types, structural delimiters, mathematical domain constraints, and handle errors predictably rather than propagating fatal crashes.